

SQL (Part-1) — In-Depth Study Guide

Complete Learning Guide: Databases, Filtering, Joins, Set Operations & Subqueries

Author: Golam Maula Lincoln

GitHub Profile: <https://github.com/gmlincoln>

Visit GitHub Profile [↗](https://github.com/gmlincoln)

1. Introduction to Databases and SQL

Before diving into SQL, it is crucial to understand the foundation of databases and relational architecture.

What is a Database?

A **database** is an organized collection of data stored and managed electronically so that it can be easily accessed, updated, and analyzed.

Benefits of Using a Database

While data can be stored in flat files (like Excel or CSV), databases provide key structural advantages:

- **Organized Storage:** Keeps data structured in well-defined schemas.
- **Quick Access:** Optimized indexing mechanisms allow high-speed retrieval.
- **Data Integrity:** Enforces constraints to prevent corrupt or duplicate records.
- **Multi-User Access:** Supports simultaneous operations by multiple users.
- **Security:** Granular access control for sensitive fields.
- **Analytics & Reporting:** Essential for dashboards, machine learning, and BI systems.

What is a Relational Database (RDBMS)?

A **Relational Database Management System (RDBMS)** stores data in two-dimensional tables (relations) consisting of **rows (tuples)** and **columns (attributes)**, linked via **Primary** and **Foreign Keys**.

What is SQL?

SQL (Structured Query Language) is the standard language used to interact with relational databases. SQL includes:

- **DQL (Data Query Language):** Retrieving data (SELECT).
- **DML (Data Manipulation Language):** Inserting, updating, and deleting records (INSERT, UPDATE, DELETE).
- **DDL (Data Definition Language):** Schema creation and management (CREATE, ALTER, DROP).

2. Advanced Filtering & Pattern Matching

SQL provides powerful clauses to filter records based on complex logical expressions and text patterns.

Logical Filtering (AND, OR, NOT)

```
-- Negating a condition with NOT
SELECT column1, column2 FROM table_name WHERE NOT condition;

-- Multiple conditions with AND
SELECT * FROM Customers WHERE Country = 'India' AND City = 'Mumbai';

-- Combining AND with nested OR conditions
SELECT * FROM Customers WHERE Country = 'America' AND (City = 'Boston' OR City = 'Chicago');
```

The DISTINCT Operator

Eliminates duplicate rows from the query output, returning only unique records.

```
SELECT DISTINCT Country, City FROM Customers;
```

Pattern Matching with the LIKE Operator

The LIKE operator evaluates pattern matches in string fields using wildcards:

- * % (Percent sign) → Represents zero, one, or multiple characters.
- * _ (Underscore) → Represents a single character.

Pattern	Description	Example Match
'a%'	Starts with "a"	"Alice", "Apple"
'%a'	Ends with "a"	"California", "Data"
'%or%'	Contains "or" in any position	"Jordan", "Actor"
'_r%'	Has "r" in the second position	"Arun", "Gram"
'a_%'	Starts with "a" and at least 2 characters long	"Am", "Aaron"
'a__%'	Starts with "a" and at least 3 characters long	"Ann", "Andrew"
'a%o'	Starts with "a" and ends with "o"	"Antonio", "Auto"

```
SELECT * FROM Employees WHERE First_Name LIKE 'J%';
```

3. SQL Joins

A JOIN clause combines records from two or more tables based on a related column between them.

1) Inner Join

Returns only records that have matching key values in both tables.

```
SELECT Customers.CustomerName, Orders.Product
FROM Customers
INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

2) Cross Join

Generates a Cartesian Product, pairing every row of the first table with every row of the second table (M times N rows).

3) Self Join

Joins a table to itself by treating it as two distinct instances using table aliases (AS). Ideal for hierarchical records (e.g., Employee to Manager).

```
SELECT e1.EmployeeName AS Employee, e2.EmployeeName AS Manager
FROM Employees AS e1
JOIN Employees AS e2 ON e1.ManagerID = e2.EmployeeID;
```

4. Set Operations

Set operations manipulate and combine query result sets based on mathematical set principles.

- **UNION:** Combines result sets and removes duplicates.
- **UNION ALL:** Combines result sets while keeping all duplicates (faster performance).
- **INTERSECT:** Returns only distinct rows present in all query outputs.
- **EXCEPT (or MINUS):** Returns distinct rows from the first query that do not exist in the second.

```
-- UNION Example
SELECT CustomerName FROM Customers
UNION
SELECT SupplierName FROM Suppliers;
```

Set Operations vs. Joins

Feature	Set Operations	Joins
Purpose	Manipulates result sets vertically based on set theory.	Combines columns from related tables horizontally based on keys.
Data Source	Result sets of independent SELECT queries.	Base tables linked by foreign keys.
Column Rule	Queries must have identical column counts and matching types.	Columns can differ in names, counts, and data types.
Operations	UNION, INTERSECT, EXCEPT	INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN

5. Subqueries (Nested Queries)

A **subquery** is a query nested within another query to evaluate intermediate values or filter criteria for the outer statement.

```
SELECT ProductName, Quantity
FROM Products
WHERE Price * Quantity > (
    SELECT AVG(Price) FROM Products
);
```

Subqueries vs. Joins Comparison

- **Subqueries:** Ideal for single-value calculations, list filtering, or sub-evaluations inside WHERE / HAVING clauses.
- **Joins:** Optimized for merging multi-column relational tables directly within the FROM clause.

✨ Keep Learning & Keep Exploring! ✨

Document created for GitHub profile: [gmlincoln \(Golam Maula Lincoln\)](#)